

Java Programming

Comprehensive Study Notes

Object-Oriented Programming & Exception Handling

● *Classes & Objects* ● *Inheritance* ● *Polymorphism*

● *Abstraction* ● *Encapsulation* ● *Interfaces*

● *Try-Catch-Finally* ● *Custom Exceptions* ● *Best Practices*

Detailed Notes with Code Examples

PART 1: Object-Oriented Programming in Java

Core Concepts, Principles & Code Examples

1.1 What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **data (objects)** rather than functions and logic. Java is a fully object-oriented language where almost everything is an object. OOP makes programs modular, reusable, and easier to maintain.

■ **Key Idea:** OOP models real-world entities as objects that have state (fields/attributes) and behaviour (methods).

The Four Pillars of OOP

Pillar	One-Line Definition
Encapsulation	Bundling data and methods; hiding internal details via access modifiers.
Inheritance	A child class acquires properties and behaviour of a parent class.
Polymorphism	One interface, many implementations — same method, different behaviour.
Abstraction	Exposing only essential features; hiding implementation complexity.

1.2 Classes and Objects

A **class** is a blueprint or template. An **object** is a concrete instance of a class created at runtime. Every object has its own copy of instance variables but shares class-level (static) members.

Anatomy of a Java Class

```
public class Student {  
    // Fields (instance variables)  
    private String name;  
    private int    rollNo;  
    private double gpa;  
  
    // Constructor  
    public Student(String name, int rollNo, double gpa) {  
        this.name    = name;  
        this.rollNo  = rollNo;  
        this.gpa     = gpa;  
    }  
  
    // Getter / Setter (Encapsulation)  
    public String getName()      { return name; }  
    public void   setName(String n) { this.name = n; }  
  
    // Behaviour  
    public void display() {
```

```

        System.out.println("Name: " + name + ", GPA: " + gpa);
    }
}

// Creating Objects
Student s1 = new Student("Ali", 101, 3.8);
Student s2 = new Student("Sara", 102, 3.9);
s1.display();

```

Constructors

A **constructor** is a special method called automatically when an object is created. It has the same name as the class and no return type.

```

// Default constructor (no-arg)
public Student() { }

// Parameterized constructor
public Student(String name, int rollNo) {
    this.name = name;
    this.rollNo = rollNo;
}

// Copy constructor
public Student(Student other) {
    this.name = other.name;
    this.rollNo = other.rollNo;
}

```

■ If you define any constructor, Java does **not** provide the default no-arg constructor automatically.

1.3 Encapsulation

Encapsulation means **wrapping data (variables) and code (methods) together** as a single unit, and **restricting direct access** to some components. This is achieved using *access modifiers*.

Access Modifier	Class	Package	Subclass	World
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

```

public class BankAccount {
    private double balance; // hidden from outside

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
}

```

```

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount; return true;
        }
        return false;
    }

    public double getBalance() { return balance; }
}

```

1.4 Inheritance

Inheritance allows a **child (sub) class** to inherit fields and methods from a **parent (super) class**, promoting code reuse. Java uses the **extends** keyword. Java supports *single* and *multilevel* inheritance for classes, but *not multiple inheritance* (use interfaces instead).

```

// Parent class
public class Animal {
    protected String name;
    public Animal(String name) { this.name = name; }
    public void eat() { System.out.println(name + " is eating."); }
    public void sleep(){ System.out.println(name + " is sleeping."); }
}

// Child class
public class Dog extends Animal {
    private String breed;
    public Dog(String name, String breed) {
        super(name); // call parent constructor
        this.breed = breed;
    }
    public void bark() { System.out.println(name + " says: Woof!"); }

    @Override
    public void eat() { // overriding parent method
        System.out.println(name + " eats dog food.");
    }
}

// Multilevel
public class GoldenRetriever extends Dog { ... }

```

Types of Inheritance in Java

Type	Description / Example
Single	One parent → one child. class B extends A
Multilevel	Chain: A → B → C. Each level adds functionality.
Hierarchical	One parent → multiple children. A → B, A → C

Type	Description / Example
Multiple	NOT supported for classes. Achieved via interfaces.
Hybrid	Combination — achieved through interfaces only.

■ **super** keyword is used to call the parent class constructor or method. **this** refers to the current instance.

1.5 Polymorphism

Polymorphism means “many forms”. In Java it allows one method/operator to behave differently based on the object or arguments.

A) Compile-Time Polymorphism — Method Overloading

Same method name, different parameter list. Resolved at **compile time**.

```
public class Calculator {
    public int    add(int a, int b)          { return a + b; }
    public double add(double a, double b)    { return a + b; }
    public int    add(int a, int b, int c)    { return a + b + c; }
    // Note: return type alone cannot differentiate overloads
}
```

B) Runtime Polymorphism — Method Overriding

Child class provides a specific implementation of a method already defined in the parent. Resolved at **runtime** via dynamic method dispatch.

```
public class Shape {
    public double area() { return 0; }
}

public class Circle extends Shape {
    private double r;
    public Circle(double r) { this.r = r; }
    @Override
    public double area() { return Math.PI * r * r; }
}

public class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) { this.w=w; this.h=h; }
    @Override
    public double area() { return w * h; }
}

// Dynamic dispatch
Shape s = new Circle(5);
System.out.println(s.area()); // calls Circle.area() at runtime
```

■ **Rules for Overriding:** Same name, same parameters, same/covariant return type. Access modifier must be $\geq$ parent's. Cannot override **static, final, or private** methods.

1.6 Abstraction

Abstraction hides implementation details and shows only **essential features**. Java achieves abstraction through **abstract classes** and **interfaces**.

Abstract Classes

```
// Cannot instantiate; may have abstract + concrete methods
public abstract class Vehicle {
    protected int speed;

    public abstract void fuelType(); // must be overridden
    public void move() {             // concrete method
        System.out.println("Vehicle moving at " + speed + " km/h");
    }
}

public class ElectricCar extends Vehicle {
    @Override
    public void fuelType() { System.out.println("Electric"); }
}
```

Interfaces

An interface is a **100% abstract** contract (prior to Java 8). Since Java 8 it can have **default** and **static** methods. Since Java 9 it can have **private** methods. A class can implement **multiple interfaces**.

```
public interface Flyable {
    int MAX_ALTITUDE = 40000; // implicitly public static final
    void fly();                // implicitly public abstract
    default void land() {
        System.out.println("Landing...");
    }
}

public interface Swimmable {
    void swim();
}

public class Duck extends Animal implements Flyable, Swimmable {
    @Override public void fly() { System.out.println("Duck flies."); }
    @Override public void swim() { System.out.println("Duck swims."); }
}
```

	Abstract Class	Interface
	Cannot	Cannot
	Abstract + Concrete	Abstract, default, static, private (Java 9+)

	Abstract Class	Interface
	Any type	public static final only
	Single (extends)	Multiple (implements)
	Yes	No
	Shared code among related classes	Unrelated classes share a contract

1.7 Static and Final Keywords

static keyword

```
public class Counter {
    private static int count = 0; // shared by all instances
    public Counter() { count++; }
    public static int getCount() { return count; } // static method
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.getCount()); // 2
```

final keyword

```
final int MAX = 100; // constant – cannot reassign
final class ImmutableClass { } // cannot be subclassed
public final void display() { // cannot be overridden
    System.out.println("Fixed behaviour");
}
```

1.8 this and super Keywords

```
public class Parent {
    int x = 10;
    public void show() { System.out.println("Parent: " + x); }
}
public class Child extends Parent {
    int x = 20;
    public void show() {
        System.out.println(this.x); // 20 – current class field
        System.out.println(super.x); // 10 – parent class field
        super.show(); // call parent method
    }
}
```

1.9 Packages and Import

Packages organize classes into namespaces, prevent naming conflicts, and control access.

```
// Declare package (first line of file)
package com.university.students;

// Import specific class
import java.util.ArrayList;

// Import all classes from package
import java.util.*;

// Fully qualified name (no import needed)
java.util.Scanner sc = new java.util.Scanner(System.in);
```


PART 2: Exception Handling in Java

Errors, Exceptions, Custom Exceptions & Best Practices

2.1 What is an Exception?

An **exception** is an unwanted or unexpected event that disrupts the normal flow of a program at runtime. Java uses a rich exception hierarchy to categorize and handle these events gracefully, preventing abrupt termination.

■ Exceptions are **objects** in Java. Every exception class is a subclass of **java.lang.Throwable**.

Exception Hierarchy

```
java.lang.Throwable
├── java.lang.Error           (JVM errors – do NOT catch)
│   ├── StackOverflowError
│   ├── OutOfMemoryError
│   └── VirtualMachineError
├── java.lang.Exception      (program errors – handle these)
│   ├── RuntimeException      (Unchecked)
│   │   ├── NullPointerException
│   │   ├── ArrayIndexOutOfBoundsException
│   │   ├── ClassCastException
│   │   ├── ArithmeticException
│   │   └── NumberFormatException
│   └── Checked Exceptions
│       ├── IOException
│       ├── FileNotFoundException
│       ├── SQLException
│       └── ClassNotFoundException
```

2.2 Checked vs Unchecked Exceptions

Type	Checked Exception	Unchecked Exception
When checked	Compile time	Runtime
Must handle?	Yes — try-catch or throws	No — optional
Superclass	Exception (not RuntimeException)	RuntimeException
Examples	IOException, SQLException	NPE, ArithmeticException
Typical cause	External resource issues	Programming bugs

2.3 try, catch, finally Blocks

The **try** block contains code that might throw an exception. The **catch** block handles the exception. The **finally** block always executes — exception or not — typically for cleanup.

```
public class DivisionExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};
        try {
            int result = numbers[1] / 0;    // ArithmeticException
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of bounds: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Generic: " + e.getMessage());
        } finally {
            System.out.println("Always runs – close resources here.");
        }
    }
}
```

■ Catch blocks are evaluated **top-to-bottom**. More specific exceptions must appear **before** general ones, or a compile error occurs.

Multi-catch (Java 7+)

```
try {
    // code
} catch (IOException | SQLException e) {
    // Handle both exception types the same way
    System.out.println("IO or SQL error: " + e.getMessage());
}
```

2.4 throw and throws Keywords

Keyword	Purpose	Syntax
throw	Explicitly throw an exception from code	throw new ExceptionType("message");
throws	Declare exceptions a method might throw (shifts responsibility to caller)	public void m() throws IOException { }

```
// throw – inside method body
public class AgeValidator {
    public static void validate(int age) {
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age: " + age);
        }
    }
}
```

```

    }
    System.out.println("Age is valid: " + age);
}
}

// throws – method signature
public void readFile(String path) throws IOException {
    FileReader fr = new FileReader(path); // may throw IOException
    // ...
}

```

2.5 try-with-resources (Java 7+)

Automatically closes resources that implement **AutoCloseable** (or **Closeable**) after the try block ends — even if an exception occurs. Eliminates verbose finally blocks for resource cleanup.

```

// Resource closed automatically – no finally needed
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("File error: " + e.getMessage());
}

// Multiple resources (closed in reverse order)
try (Connection con = DriverManager.getConnection(url);
     Statement st = con.createStatement()) {
    // ...
}

```

2.6 Custom (User-Defined) Exceptions

You can create your own exception classes by extending **Exception** (checked) or **RuntimeException** (unchecked). Always provide at least: a no-arg constructor, a message constructor, and optionally a cause constructor.

```

// Custom Checked Exception
public class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds. Shortage: " + amount);
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

// Custom Unchecked Exception

```

```

public class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
    public InvalidAgeException(String message, Throwable cause) {
        super(message, cause);
    }
}

// Using custom exception
public class BankAccount {
    private double balance = 1000;

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
    }
}

// Calling code
BankAccount acc = new BankAccount();
try {
    acc.withdraw(1500);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage());
    System.out.printf("Short by: %.2f%n", e.getAmount());
}

```

2.7 Exception Propagation

When an exception is not caught in the current method, it **propagates up the call stack** until it is caught or reaches the JVM (which then terminates the program with a stack trace).

```

public class Propagation {
    static void method3() { int x = 10/0; }           // throws here
    static void method2() { method3(); }             // propagates
    static void method1() { method2(); }             // propagates

    public static void main(String[] args) {
        try {
            method1();
        } catch (ArithmeticException e) {            // caught here
            System.out.println("Caught: " + e.getMessage());
        }
    }
}

```

2.8 Key Throwable Methods

Method	Description
<code>getMessage()</code>	Returns the detail message string of the exception.
<code>getCause()</code>	Returns the cause (another Throwable) or null.
<code>toString()</code>	Returns class name + <code>getMessage()</code> .
<code>printStackTrace()</code>	Prints the stack trace to stderr (useful for debugging).
<code>getStackTrace()</code>	Returns array of <code>StackTraceElement</code> objects.
<code>initCause()</code>	Sets the cause (exception chaining).

2.9 Exception Chaining

Exception chaining allows wrapping a lower-level exception inside a higher-level one, preserving the full diagnostic chain.

```
public void connectToDatabase() throws DatabaseException {
    try {
        // low-level operation
        Class.forName("com.mysql.Driver");
    } catch (ClassNotFoundException e) {
        // wrap and rethrow with context
        throw new DatabaseException("Driver not found", e);
    }
}

// Catching and inspecting
try {
    connectToDatabase();
} catch (DatabaseException e) {
    System.out.println("DB Error: " + e.getMessage());
    System.out.println("Caused by: " + e.getCause().getMessage());
}
```

2.10 Best Practices for Exception Handling

- **Catch specific exceptions** first, then general ones. Never catch `Exception` or `Throwable` blindly unless absolutely necessary.
- **Never swallow exceptions silently.** An empty catch block hides bugs. At minimum, log the error.
- **Use try-with-resources** for all `Closeable`/`AutoCloseable` resources (files, streams, DB connections).
- **Prefer unchecked exceptions** (`RuntimeException` subclasses) for programming errors; use checked exceptions for recoverable external failures.
- **Provide meaningful messages** in custom exceptions so debugging is straightforward.
- **Do not use exceptions for flow control.** Use if-else for expected conditions.
- **Document exceptions** with `@throws` Javadoc tags on public methods.
- **Log before rethrowing** to maintain context: log the cause, then wrap and rethrow.

- **Clean up resources in finally** (or use try-with-resources) to avoid leaks.
- **Exception chaining** — always pass the original cause when wrapping to preserve the stack trace.

■ **Common Anti-Pattern:** `catch (Exception e) {}` — *never do this. Always handle or log.*

2.11 Quick Reference Cheat Sheet

Scenario	Solution
Might throw checked exception	Surround with try-catch OR add throws to signature
Want to propagate	Declare with throws; do not catch
Always-run cleanup	Use finally block
File / stream handling	Use try-with-resources
Domain-specific error	Create custom Exception subclass
Wrap low-level exception	Exception chaining — pass cause to constructor
Multiple exception types same block	Multi-catch: <code>catch (A B e)</code>
Log & rethrow	<code>log(e); throw e;</code> or <code>throw new WrapperEx(msg, e)</code>

Summary: OOP & Exception Handling at a Glance

OOP Core Concepts Summary

Concept	Key Points
Class & Object	Class = blueprint; Object = instance with state & behaviour
Constructor	Same name as class, no return type; overloadable
Encapsulation	private fields + public getters/setters; data hiding
Inheritance	extends keyword; super(); method overriding with @Override
Polymorphism	Overloading (compile-time) + Overriding (runtime dispatch)
Abstraction	abstract class (partial) vs interface (contract, multiple impl.)
static	Belongs to class, not instance; shared across all objects
final	final variable = constant; final method = no override; final class = no subclass
Interface (Java 8+)	abstract, default, static, private methods; multiple implementation

Exception Handling Summary

Concept	Key Points
Throwable	Root; two children: Error (JVM) and Exception (program)
Checked	Compile-time; must handle; extends Exception
Unchecked	Runtime; optional handle; extends RuntimeException
try-catch-finally	try: risky code; catch: handle; finally: always runs
throw	Manually throw an exception: throw new XxxException()
throws	Declare method might throw: public void m() throws XxxEx
try-with-resources	Auto-close AutoCloseable; eliminates finally for cleanup
Custom Exception	extend Exception (checked) or RuntimeException (unchecked)
Exception Chaining	new WrapperEx(msg, cause) — preserves original stack trace
Multi-catch	catch (A B e) — Java 7+; handle multiple types same block

■ **Exam Tips:** (1) Remember: Error should not be caught; Exception should be handled. (2) finally always runs — even after a return statement in try. (3) Method overloading is resolved at compile time; overriding at runtime. (4) An abstract class can have constructors; an interface cannot. (5) A class can implement multiple interfaces but extend only one class.